

Security Scan Report

Target: C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberSceurity
Project - Copy\security-signal-engine\test_vulnerable_app

Scan Date: July 23, 2026 at 14:48 UTC

Scan ID: 72ff03fe-3673-4122-86e6-da343b9af6a7

0 CRITICAL

2 HIGH

3 MEDIUM

4 LOW

Executive Summary

Security scan of 'C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberSceurity Project - Copy\security-signal-engine\test_vulnerable_app' identified 39 raw findings, reduced to 9 unique issues after deduplication. ■ 2 HIGH-severity issue(s) should be prioritized for remediation within the next 48 hours. 3 MEDIUM-priority issue(s) should be addressed in the current development sprint. 4 LOW-priority issue(s) have been logged for backlog tracking.

Scan Statistics

Metric	Value
Total Raw Findings	39
After Deduplication	9
Critical Findings	0
High Findings	2
Medium Findings	3
Low Findings	4
Scan Duration	9.1s

Detailed Findings

[HIGH] #1: Ssrf in C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberSceurity Project - Copy\security-signal-engine\test_vulnerable_app\vulnerable_server.py:310

Risk Score	0.6174
Affected Location	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberSceurity Project - Copy\security-signal-engine\te
Vulnerability Type	SSRF
CWE ID	CWE-918: Server-Side Request Forgery (SSRF)
Detected By	semgrep

Explanation
Server-Side Request Forgery (SSRF) occurs when an application makes HTTP requests to URLs controlled by user input, allowing attackers to access internal services and infrastructure.

Code or HTTP Excerpt

```
308 | def fetch_url():
309 |     """■ SSRF: Server fetches any URL provided by the user."""
>>> 310 |     url = request.args.get("url", "")
      311 |     content = ""
      312 |     <h2>URL Fetcher</h2>
```

Business Impact
An attacker could access internal services, cloud metadata endpoints, or other systems not intended to be publicly accessible.

Fix Recommendation
Rule-based recommendation: Whitelist allowed domains and IP addresses for outbound requests. Block requests to internal IP ranges (169.254.x.x, 10.x.x.x, 172.16-31.x.x, 192.168.x.x). Validate and sanitize all URLs before making requests.
LLM recommendation: Whitelist allowed domains and IP addresses for outbound requests. Block requests to internal IP ranges (169.254.x.x, 10.x.x.x, 172.16-31.x.x, 192.168.x.x). Validate and sanitize all URLs before making requests.

[MEDIUM] #2: Open Redirect in C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberSceurity Project - Copy\security-signal-engine\test_vulnerable_app\vulnerable_server.py:298

Risk Score	0.4508
Affected Location	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberSceurity Project - Copy\security-signal-engine\te
Vulnerability Type	OPEN_REDIRECT
CWE ID	CWE-601: URL Redirection to Untrusted Site ('Open Redirect')
Detected By	semgrep

Explanation

Open redirect vulnerabilities allow attackers to redirect users to malicious websites by manipulating URL parameters, which can be used for phishing attacks.

Code or HTTP Excerpt

```
296 | def open_redirect():
297 |     """■ Open Redirect: Redirects to any URL provided by the user."""
>>> 298 |     url = request.args.get("url", "/")
299 |     # ■ VULNERABLE: No validation of redirect target
300 |     return redirect(url)
```

Business Impact

Attackers could use your application to redirect users to phishing sites, potentially stealing credentials.

Fix Recommendation

Rule-based recommendation: Validate redirect URLs against a whitelist of allowed domains. Use relative paths instead of full URLs for internal redirects. Reject any redirect URLs pointing to external domains.

LLM recommendation: Validate redirect URLs against a whitelist of allowed domains. Use relative paths instead of full URLs for internal redirects. Reject any redirect URLs pointing to external domains.

[MEDIUM] #3: Unknown in C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\test_vulnerable_app\vulnerable_server.py:463

Risk Score	0.4508
Affected Location	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\te
Vulnerability Type	UNKNOWN
CWE ID	CWE-611: Improper Restriction of XML External Entity Reference
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
461 |         return render_template_string(LAYOUT, title="XML API", content=content)
462 |
>>> 463 |     import xml.etree.ElementTree as ET
464 |     try:
465 |         # ■ VULNERABLE: Parsing XML without disabling external entities
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[HIGH] #4: Sql Injection in C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\test_vulnerable_app\vulnerable_server.py:160

Risk Score	0.4324
Affected Location	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\te
Vulnerability Type	SQL_INJECTION
CWE ID	CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Detected By	semgrep

Explanation
SQL Injection occurs when user-controlled input is directly concatenated into SQL queries, allowing an attacker to manipulate the query logic. This can lead to unauthorized data access, modification, or deletion of database records.

Code or HTTP Excerpt

```
158 | def users():
159 |     """■ SQL Injection: User input directly concatenated into SQL."""
>>> 160 |     user_id = request.args.get("id", "")
161 |     content = "<h2>User Lookup</h2>"
162 |     content += '<form action="/users" method="get">'
```

Business Impact
An attacker could read, modify, or delete all data in the database, potentially exposing customer records and sensitive business data.

Fix Recommendation
Rule-based recommendation: Use parameterized queries instead of string concatenation. In Python, use placeholder syntax: `cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,))`. For ORMs, use the built-in query builder instead of raw SQL.
LLM recommendation: Use parameterized queries instead of string concatenation. In Python, use placeholder syntax: `cursor.execute('SELECT * FROM users WHERE id = ?', (user_id,))`. For ORMs, use the built-in query builder instead of raw SQL.

[MEDIUM] #5: Unknown in C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\test_vulnerable_app\vulnerable_server.py:149

Risk Score	0.4230
Affected Location	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\te
Vulnerability Type	UNKNOWN
CWE ID	CWE-614: Sensitive Cookie in HTTPS Session Without 'Secure' Attribute
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
147 |      # ■ Insecure cookie
148 |      resp.set_cookie("session_id", "abc123", httponly=False, secure=False, samesite=None)
>>> 149 |      resp.set_cookie("user_token", "eyJhbGciOiJIUzI1NiJ9.test", httponly=False)
150 |      return resp
151 |
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[LOW] #6: Unknown in C:\Users\Dhruv\Desktop\College

Stuff\Projects\CyberScurity Project -

Copy\security-signal-engine\test_vulnerable_app\vulnerable_server.py:171

Risk Score	0.3920
Affected Location	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\te
Vulnerability Type	UNKNOWN
CWE ID	CWE-704: Incorrect Type Conversion or Cast
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
169 |      try:
170 |          # ■ VULNERABLE: Direct string concatenation in SQL
>>> 171 |          query = f"SELECT * FROM users WHERE id = {user_id}"
172 |          cursor.execute(query)
173 |          rows = cursor.fetchall()
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[LOW] #7: Xss in C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\test_vulnerable_app\vulnerable_server.py:163

Risk Score	0.3478
Affected Location	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\te
Vulnerability Type	XSS
CWE ID	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
Detected By	semgrep

Explanation
Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output without proper encoding, allowing attackers to inject malicious scripts that execute in users' browsers.

Code or HTTP Excerpt

```
161 |     content = "<h2>User Lookup</h2>"
162 |     content += '<form action="/users" method="get">'
>>> 163 |     content += '<input name="id" placeholder="Enter user ID" value="' + user_id + '">'
164 |     content += '<button type="submit">Search</button></form>'
165 |
```

Business Impact
An attacker could steal user session tokens, redirect users to malicious sites, or perform actions on behalf of authenticated users.

Fix Recommendation
Rule-based recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

LLM recommendation: Apply context-aware output encoding to all user-controlled data before rendering in HTML. Use your framework's built-in escaping (e.g., Jinja2 auto-escaping, React JSX). For manual escaping, reference OWASP ESAPI.

[LOW] #8: Unknown in C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\test_vulnerable_app\vulnerable_server.py:124

Risk Score	0.3384
Affected Location	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\te
Vulnerability Type	UNKNOWN
CWE ID	CWE-96: Improper Neutralization of Directives in Statically Saved Code ('Static Code Injection')
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
122 | # ■ Missing security headers (X-Frame-Options, CSP, etc.)
123 | # ■ Server version disclosure
>>> 124 | resp = make_response(render_template_string(LAYOUT, title="Home", content="")
125 |     <p>Welcome to VulnApp – an intentionally vulnerable application.</p>
126 |     <h2>Navigation</h2>
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

[LOW] #9: Unknown in C:\Users\Dhruv\Desktop\College

Stuff\Projects\CyberScurity Project -

Copy\security-signal-engine\test_vulnerable_app\vulnerable_server.py:321

Risk Score	0.3384
Affected Location	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\te
Vulnerability Type	UNKNOWN
CWE ID	CWE-939: Improper Authorization in Handler for Custom URL Scheme
Detected By	semgrep

Explanation

A security vulnerability was detected in your code. This issue could potentially be exploited by attackers to compromise the security of your application.

Code or HTTP Excerpt

```
319 | try:
320 |     # ■ VULNERABLE: No URL validation, allows internal network access
>>> 321 | resp = urllib.request.urlopen(url, timeout=5)
322 |     body = resp.read(4096).decode("utf-8", errors="replace")
323 |     content += f"<h3>Response from {url}</h3>"
```

Business Impact

This vulnerability could be exploited to compromise application security and potentially access sensitive data.

Fix Recommendation

Rule-based recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

LLM recommendation: Review the flagged code and apply security best practices. Consult the OWASP guidelines for specific remediation steps relevant to this type of vulnerability.

Appendix

Deduplication Statistics

Total raw findings: 39 | After deduplication: 9 | Reduction: 77%

Scan Metadata

Scan ID	72ff03fe-3673-4122-86e6-da343b9af6a7
Target	C:\Users\Dhruv\Desktop\College Stuff\Projects\CyberScurity Project - Copy\security-signal-engine\te
Timestamp	2026-07-23T14:48:02.003615+00:00
Duration	9.1s
Scanners Used	semgrep
Scanner Versions	semgrep=1.159.0

Raw Findings Data (JSON)

```
[
  {
    "id": "4e9344d0-c430-4f92-add2-817160e67651",
    "title": "Ssrf in C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\secu",
    "file": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-sig",
    "line_start": 310,
    "line_end": 326,
    "severity": "HIGH",
    "vuln_type": "SSRF",
    "description": "Data from request object is passed to a new server-side request. This could lead to a s",
    "rule_id": "python.django.security.injection.ssrfs.ssrfinjectionurllib.ssrfinjectionurllib",
    "confidence": 0.9,
    "cwe_id": "CWE-918: Server-Side Request Forgery (SSRF)",
    "cvss_score": null,
    "code_snippet": "
        308 | def fetch_url():\n
        309 |         \"\"\"u274c SSRF: Server fetches any URL p
    "sources": [
      "semgrep"
    ],
    "risk_score": 0.6174,
    "risk_category": "HIGH",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Server-Side Request Forgery (SSRF) occurs when an application makes HTTP requests to",
    "fix_suggestion": "Whitelist allowed domains and IP addresses for outbound requests. Block requests to",
    "business_impact": "An attacker could access internal services, cloud metadata endpoints, or other syste",
    "raw": {
      "check_id": "python.django.security.injection.ssrfs.ssrfinjectionurllib.ssrfinjectionurllib",
      "path": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-s",
      "start": {
        "line": 310,
        "col": 5,
        "offset": 13511
      },
      "end": {
        "line": 326,
        "col": 68,
        "offset": 14219
      },
      "extra": {
```

```

"message": "Data from request object is passed to a new server-side request. This could lead to a s
"metadata": {
  "cwe": [
    "CWE-918: Server-Side Request Forgery (SSRF)"
  ],
  "owasp": [
    "A10:2021 - Server-Side Request Forgery (SSRF)",
    "A01:2025 - Broken Access Control"
  ],
  "references": [
    "https://owasp.org/www-community/attacks/Server_Side_Request_Forgery"
  ],
  "category": "security",
  "technology": [
    "django"
  ],
  "cwe2022-top25": true,
  "cwe2021-top25": true,
  "subcategory": [
    "vuln"
  ],
  "likelihood": "MEDIUM",
  "impact": "HIGH",
  "confidence": "MEDIUM",
  "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
  "vulnerability_class": [
    "Server-Side Request Forgery (SSRF)"
  ],
  "source": "https://semgrep.dev/r/python.django.security.injection.ssrp.ssrp-injection-urllib.ssrp",
  "shortlink": "https://sg.run/6n2B"
},
"severity": "ERROR",
"fingerprint": "requires login",
"lines": "requires login",
"validation_state": "NO_VALIDATOR",
"engine_kind": "OSS"
}
},
{
  "id": "bec541a7-a0c7-48d1-8b69-f643ea5998a0",
  "title": "Open Redirect in C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - C
  "file": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-sign
  "line_start": 298,
  "line_end": 300,
  "severity": "HIGH",
  "vuln_type": "OPEN_REDIRECT",
  "description": "Data from request is passed to redirect(). This is an open redirect and could be exploit
  "rule_id": "python.flask.security.open-redirect.open-redirect",
  "confidence": 0.9,
  "cwe_id": "CWE-601: URL Redirection to Untrusted Site ('Open Redirect')",
  "cvss_score": null,
  "code_snippet": "    296 | def open_redirect():\n    297 |         \"\"\"Open Redirect: Redirects t
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.4508,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "Open redirect vulnerabilities allow attackers to redirect users to malicious website
  "fix_suggestion": "Validate redirect URLs against a whitelist of allowed domains. Use relative paths ins
  "business_impact": "Attackers could use your application to redirect users to phishing sites, potential
  "raw": {
    "check_id": "python.flask.security.open-redirect.open-redirect",
    "path": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-s
    "start": {
      "line": 298,
      "col": 5,
      "offset": 12886
    },
  },

```

```

"end": {
  "line": 300,
  "col": 25,
  "offset": 13000
},
"extra": {
  "message": "Data from request is passed to redirect(). This is an open redirect and could be exploited",
  "metadata": {
    "cwe": [
      "CWE-601: URL Redirection to Untrusted Site ('Open Redirect')"
    ],
    "owasp": [
      "A01:2021 - Broken Access Control",
      "A01:2025 - Broken Access Control"
    ],
    "references": [
      "https://flask-login.readthedocs.io/en/latest/#login-example",
      "https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html",
      "https://docs.python.org/3/library/urllib.parse.html#url-parsing"
    ],
    "category": "security",
    "technology": [
      "flask"
    ],
    "subcategory": [
      "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
      "Open Redirect"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.open-redirect.open-redirect",
    "shortlink": "https://sg.run/kXe2"
  },
  "severity": "ERROR",
  "fingerprint": "requires login",
  "lines": "requires login",
  "validation_state": "NO_VALIDATOR",
  "engine_kind": "OSS"
}
},
{
  "id": "687ca1d8-f2bf-4b53-a0a0-9770afe4547d",
  "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-signature.py",
  "file": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-signature.py",
  "line_start": 463,
  "line_end": 463,
  "severity": "HIGH",
  "vuln_type": "UNKNOWN",
  "description": "The Python documentation recommends using `defusedxml` instead of `xml` because the native `xml` module is vulnerable to XML External Entity (XXE) attacks.",
  "rule_id": "python.lang.security.use-defused-xml.use-defused-xml",
  "confidence": 0.9,
  "cwe_id": "CWE-611: Improper Restriction of XML External Entity Reference",
  "cvss_score": null,
  "code_snippet": "461 |         return render_template_string(LAYOUT, title=\"XML API\", content=content)"
  "sources": [
    "semgrep"
  ],
  "risk_score": 0.4508,
  "risk_category": "MEDIUM",
  "category_source": "score",
  "category_reason": "score-threshold",
  "llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be exploited to compromise application security and potentially leak sensitive data.",
  "fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelines for XXE prevention.",
  "business_impact": "This vulnerability could be exploited to compromise application security and potentially leak sensitive data.",
  "raw": {
    "check_id": "python.lang.security.use-defused-xml.use-defused-xml",

```

```

"path": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-sig",
"start": {
  "line": 463,
  "col": 5,
  "offset": 20271
},
"end": {
  "line": 463,
  "col": 39,
  "offset": 20305
},
"extra": {
  "message": "The Python documentation recommends using `defusedxml` instead of `xml` because the nat",
  "metadata": {
    "owasp": [
      "A04:2017 - XML External Entities (XXE)",
      "A05:2021 - Security Misconfiguration",
      "A02:2025 - Security Misconfiguration"
    ],
    "cwe": [
      "CWE-611: Improper Restriction of XML External Entity Reference"
    ],
    "references": [
      "https://docs.python.org/3/library/xml.html",
      "https://github.com/tiran/defusedxml",
      "https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processing"
    ],
    "category": "security",
    "technology": [
      "python"
    ],
    "cwe2022-top25": true,
    "cwe2021-top25": true,
    "subcategory": [
      "audit"
    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
      "XML Injection"
    ],
    "source": "https://semgrep.dev/r/python.lang.security.use-defused-xml.use-defused-xml",
    "shortlink": "https://sg.run/kX47"
  },
  "severity": "ERROR",
  "fingerprint": "requires login",
  "lines": "requires login",
  "validation_state": "NO_VALIDATOR",
  "engine_kind": "OSS"
}
},
{
  "id": "bffe1293-87c4-4ad8-94ac-0e68e6c493d4",
  "title": "Sql Injection in C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - C",
  "file": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-sig",
  "line_start": 160,
  "line_end": 185,
  "severity": "MEDIUM",
  "vuln_type": "SQL_INJECTION",
  "description": "User-controlled data from a request is passed to 'execute()'. This could lead to a SQL i",
  "rule_id": "python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection-db",
  "confidence": 0.7,
  "cwe_id": "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')",
  "cvss_score": null,
  "code_snippet": "      158 | def users():\n      159 |         \"\"\"SQL Injection: User input directly",
  "sources": [
    "semgrep"
  ],

```

```

    "risk_score": 0.4324,
    "risk_category": "HIGH",
    "category_source": "vuln_floor",
    "category_reason": "minimum floor for SQL_INJECTION",
    "llm_explanation": "SQL Injection occurs when user-controlled input is directly concatenated into SQL queries.",
    "fix_suggestion": "Use parameterized queries instead of string concatenation. In Python, use placeholders.",
    "business_impact": "An attacker could read, modify, or delete all data in the database, potentially exposing sensitive information.",
    "raw": {
      "check_id": "python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injection",
      "path": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-signatures\\",
      "start": {
        "line": 160,
        "col": 5,
        "offset": 6608
      },
      "end": {
        "line": 185,
        "col": 21,
        "offset": 7824
      },
      "extra": {
        "message": "User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection.",
        "metadata": {
          "cwe": [
            "CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')"
          ],
          "owasp": [
            "A01:2017 - Injection",
            "A03:2021 - Injection",
            "A05:2025 - Injection"
          ],
          "references": [
            "https://docs.djangoproject.com/en/3.0/topics/security/#sql-injection-protection"
          ],
          "category": "security",
          "technology": [
            "django"
          ],
          "cwe2022-top25": true,
          "cwe2021-top25": true,
          "subcategory": [
            "vuln"
          ],
          "likelihood": "MEDIUM",
          "impact": "HIGH",
          "confidence": "MEDIUM",
          "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
          "vulnerability_class": [
            "SQL Injection"
          ],
          "source": "https://semgrep.dev/r/python.django.security.injection.sql.sql-injection-using-db-cursor-execute",
          "shortlink": "https://sg.run/qx7y"
        },
        "severity": "WARNING",
        "fingerprint": "requires login",
        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
      }
    }
  },
  {
    "id": "d03aa4f5-d26a-4fad-af70-018b8f83b76c",
    "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-signatures\\",
    "file": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-signatures\\",
    "line_start": 149,
    "line_end": 149,
    "severity": "MEDIUM",
    "vuln_type": "UNKNOWN",
    "description": "Found a Flask cookie with insecurely configured properties. By default the secure, httponly, and samesite attributes should be set to protect cookies from being accessed by scripts and other clients.",
    "rule_id": "python.flask.security.audit.secure-set-cookie.secure-set-cookie",
  }
}

```

```

"confidence": 0.7,
"cwe_id": "CWE-614: Sensitive Cookie in HTTPS Session Without 'Secure' Attribute",
"cvss_score": null,
"code_snippet": "      147 |      # \u274c Insecure cookie\n      148 |      resp.set_cookie(\"session_id\",
\"sources\": [
    \"semgrep\"
],
\"risk_score\": 0.423,
\"risk_category\": \"MEDIUM\",
\"category_source\": \"score\",
\"category_reason\": \"score-threshold\",
\"llm_explanation\": \"A security vulnerability was detected in your code. This issue could potentially be
\"fix_suggestion\": \"Review the flagged code and apply security best practices. Consult the OWASP guidelin
\"business_impact\": \"This vulnerability could be exploited to compromise application security and potent
\"raw\": {
  \"check_id\": \"python.flask.security.audit.secure-set-cookie.secure-set-cookie\",
  \"path\": \"C:\\\\Users\\\\Dhruv\\\\Desktop\\\\College Stuff\\\\Projects\\\\CyberScurity Project - Copy\\\\security-s
  \"start\": {
    \"line\": 149,
    \"col\": 5,
    \"offset\": 5991
  },
  \"end\": {
    \"line\": 149,
    \"col\": 79,
    \"offset\": 6065
  },
  \"extra\": {
    \"message\": \"Found a Flask cookie with insecurely configured properties. By default the secure, http
    \"metadata\": {
      \"cwe\": [
        \"CWE-614: Sensitive Cookie in HTTPS Session Without 'Secure' Attribute\"
      ],
      \"owasp\": [
        \"A05:2021 - Security Misconfiguration\",
        \"A02:2025 - Security Misconfiguration\"
      ],
      \"references\": [
        \"https://flask.palletsprojects.com/en/3.0.x/api/#flask.Response.set_cookie\",
        \"https://flask.palletsprojects.com/en/3.0.x/security/#set-cookie-options\"
      ],
      \"category\": \"security\",
      \"technology\": [
        \"python\",
        \"flask\"
      ],
      \"subcategory\": [
        \"audit\"
      ],
      \"likelihood\": \"LOW\",
      \"impact\": \"LOW\",
      \"confidence\": \"LOW\",
      \"functional-categories\": [
        \"web::search::cookie-config::flask\"
      ],
      \"license\": \"Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license\",
      \"vulnerability_class\": [
        \"Cookie Security\"
      ],
      \"source\": \"https://semgrep.dev/r/python.flask.security.audit.secure-set-cookie.secure-set-cookie\",
      \"shortlink\": \"https://sg.run/gLkZ\"
    },
    \"severity\": \"WARNING\",
    \"fingerprint\": \"requires login\",
    \"lines\": \"requires login\",
    \"validation_state\": \"NO_VALIDATOR\",
    \"engine_kind\": \"OSS\"
  }
}
},
{

```

```

"id": "f5654b09-062d-408e-93d2-1dc144244c75",
"title": "Unknown in C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\s",
"file": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-sign",
"line_start": 171,
"line_end": 171,
"severity": "HIGH",
"vuln_type": "UNKNOWN",
"description": "Detected user input used to manually construct a SQL string. This is usually bad practi",
"rule_id": "python.flask.security.injection.tainted-sql-string.tainted-sql-string",
"confidence": 0.9,
"cwe_id": "CWE-704: Incorrect Type Conversion or Cast",
"cvss_score": null,
"code_snippet": "    169 |          try:\\n    170 |          # \\u274c VULNERABLE: Direct string con",
"sources": [
    "semgrep"
],
"risk_score": 0.392,
"risk_category": "LOW",
"category_source": "score",
"category_reason": "score-threshold",
"llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be",
"fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelin",
"business_impact": "This vulnerability could be exploited to compromise application security and potent",
"raw": {
    "check_id": "python.flask.security.injection.tainted-sql-string.tainted-sql-string",
    "path": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-s",
    "start": {
        "line": 171,
        "col": 21,
        "offset": 7075
    },
    "end": {
        "line": 171,
        "col": 64,
        "offset": 7118
    },
    "extra": {
        "message": "Detected user input used to manually construct a SQL string. This is usually bad practi",
        "metadata": {
            "cwe": [
                "CWE-704: Incorrect Type Conversion or Cast"
            ],
            "owasp": [
                "A01:2017 - Injection",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "references": [
                "https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-textual-sql",
                "https://www.tutorialspoint.com/sqlalchemy/sqlalchemy_quick_guide.htm",
                "https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-more-specific-text-with-table-express",
            ],
            "category": "security",
            "technology": [
                "sqlalchemy",
                "flask"
            ],
            "subcategory": [
                "vuln"
            ],
            "impact": "MEDIUM",
            "likelihood": "MEDIUM",
            "confidence": "MEDIUM",
            "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
            "vulnerability_class": [
                "Improper Validation"
            ],
            "source": "https://semgrep.dev/r/python.flask.security.injection.tainted-sql-string.tainted-sql-s",
            "shortlink": "https://sg.run/JxZj"
        },
        "severity": "ERROR",

```

```

        "fingerprint": "requires login",
        "lines": "requires login",
        "validation_state": "NO_VALIDATOR",
        "engine_kind": "OSS"
    }
}
},
{
    "id": "586c9969-4c15-47f9-ac62-853bf1be8e9b",
    "title": "Xss in C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-sign",
    "file": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-sign",
    "line_start": 163,
    "line_end": 163,
    "severity": "MEDIUM",
    "vuln_type": "XSS",
    "description": "Detected user input flowing into a manually constructed HTML string. You may be accident",
    "rule_id": "python.django.security.injection.raw-html-format.raw-html-format",
    "confidence": 0.7,
    "cwe_id": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    "cvss_score": null,
    "code_snippet": "        161 |         content = \"<h2>User Lookup</h2>\"\\n        162 |         content += '<form act",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.3478,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "Cross-Site Scripting (XSS) occurs when untrusted data is included in web page output",
    "fix_suggestion": "Apply context-aware output encoding to all user-controlled data before rendering in H",
    "business_impact": "An attacker could steal user session tokens, redirect users to malicious sites, or p",
    "raw": {
        "check_id": "python.django.security.injection.raw-html-format.raw-html-format",
        "path": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-s",
        "start": {
            "line": 163,
            "col": 16,
            "offset": 6750
        },
        "end": {
            "line": 163,
            "col": 87,
            "offset": 6821
        },
    },
    "extra": {
        "message": "Detected user input flowing into a manually constructed HTML string. You may be accident",
        "metadata": {
            "cwe": [
                "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
            ],
            "owasp": [
                "A07:2017 - Cross-Site Scripting (XSS)",
                "A03:2021 - Injection",
                "A05:2025 - Injection"
            ],
            "category": "security",
            "technology": [
                "django"
            ],
            "references": [
                "https://docs.djangoproject.com/en/3.2/topics/http/shortcuts/#render",
                "https://docs.djangoproject.com/en/3.2/topics/security/#cross-site-scripting-xss-protection"
            ],
            "cwe2022-top25": true,
            "cwe2021-top25": true,
            "subcategory": [
                "vuln"
            ],
            "likelihood": "HIGH",
            "impact": "MEDIUM",
            "confidence": "MEDIUM",

```

```

        "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
        "vulnerability_class": [
            "Cross-Site-Scripting (XSS)"
        ],
        "source": "https://semgrep.dev/r/python.django.security.injection.raw-html-format.raw-html-format",
        "shortlink": "https://sg.run/oYj1"
    },
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
}
}
},
{
    "id": "8d2b3658-da16-47bf-9ae3-27d931845e78",
    "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\",
    "file": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-sign",
    "line_start": 124,
    "line_end": 143,
    "severity": "MEDIUM",
    "vuln_type": "UNKNOWN",
    "description": "Found a template created with string formatting. This is susceptible to server-side temp",
    "rule_id": "python.flask.security.audit.render-template-string.render-template-string",
    "confidence": 0.7,
    "cwe_id": "CWE-96: Improper Neutralization of Directives in Statically Saved Code ('Static Code Injection)",
    "cvss_score": null,
    "code_snippet": "        122 |          # \u274c Missing security headers (X-Frame-Options, CSP, etc.)\n        123 |",
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.3384,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be",
    "fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelin",
    "business_impact": "This vulnerability could be exploited to compromise application security and potent",
    "raw": {
        "check_id": "python.flask.security.audit.render-template-string.render-template-string",
        "path": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-s",
        "start": {
            "line": 124,
            "col": 26,
            "offset": 4583
        },
        "end": {
            "line": 143,
            "col": 9,
            "offset": 5708
        },
        "extra": {
            "message": "Found a template created with string formatting. This is susceptible to server-side temp",
            "metadata": {
                "cwe": [
                    "CWE-96: Improper Neutralization of Directives in Statically Saved Code ('Static Code Injection)",
                ],
                "owasp": [
                    "A03:2021 - Injection",
                    "A05:2025 - Injection"
                ],
                "references": [
                    "https://nvisium.com/blog/2016/03/09/exploring-ssti-in-flask-jinja2.html"
                ],
                "category": "security",
                "technology": [
                    "flask"
                ],
                "subcategory": [
                    "audit"
                ]
            }
        }
    }
}

```

```

    ],
    "likelihood": "LOW",
    "impact": "MEDIUM",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
        "Code Injection"
    ],
    "source": "https://semgrep.dev/r/python.flask.security.audit.render-template-string.render-template-string",
    "shortlink": "https://sg.run/8yjE"
},
{
    "severity": "WARNING",
    "fingerprint": "requires login",
    "lines": "requires login",
    "validation_state": "NO_VALIDATOR",
    "engine_kind": "OSS"
}
}
},
{
    "id": "028bfc88-40b3-4287-9fe9-bed24890d8e9",
    "title": "Unknown in C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-signature.py",
    "file": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-signature.py",
    "line_start": 321,
    "line_end": 321,
    "severity": "MEDIUM",
    "vuln_type": "UNKNOWN",
    "description": "Detected a dynamic value being used with urllib. urllib supports 'file://' schemes, so a",
    "rule_id": "python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected",
    "confidence": 0.7,
    "cwe_id": "CWE-939: Improper Authorization in Handler for Custom URL Scheme",
    "cvss_score": null,
    "code_snippet": "
        319 |         try:
        320 |             # \u274c VULNERABLE: No URL validation,
    "sources": [
        "semgrep"
    ],
    "risk_score": 0.3384,
    "risk_category": "LOW",
    "category_source": "score",
    "category_reason": "score-threshold",
    "llm_explanation": "A security vulnerability was detected in your code. This issue could potentially be",
    "fix_suggestion": "Review the flagged code and apply security best practices. Consult the OWASP guidelin",
    "business_impact": "This vulnerability could be exploited to compromise application security and potent",
    "raw": {
        "check_id": "python.lang.security.audit.dynamic-urllib-use-detected.dynamic-urllib-use-detected",
        "path": "C:\\Users\\Dhruv\\Desktop\\College Stuff\\Projects\\CyberScurity Project - Copy\\security-signature.py",
        "start": {
            "line": 321,
            "col": 20,
            "offset": 13906
        },
        "end": {
            "line": 321,
            "col": 58,
            "offset": 13944
        },
        "extra": {
            "message": "Detected a dynamic value being used with urllib. urllib supports 'file://' schemes, so a",
            "metadata": {
                "cwe": [
                    "CWE-939: Improper Authorization in Handler for Custom URL Scheme"
                ],
                "owasp": "A01:2017 - Injection",
                "source-rule-url": "https://github.com/PyCQA/bandit/blob/b1411bfb43795d3fffd268bef17a839dee954c2b1",
                "bandit-code": "B310",
                "asvs": {
                    "control_id": "5.2.4 Dynamic Code Execution Features",
                    "control_url": "https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-Sanitization",
                    "section": "V5: Validation, Sanitization and Encoding Verification Requirements",
                    "version": "4"
                }
            },
        }
    },
}

```

```
    "category": "security",
    "technology": [
      "python"
    ],
    "references": [
      "https://cwe.mitre.org/data/definitions/939.html"
    ],
    "subcategory": [
      "audit"
    ],
    "likelihood": "LOW",
    "impact": "LOW",
    "confidence": "LOW",
    "license": "Semgrep Rules License v1.0. For more details, visit semgrep.dev/legal/rules-license",
    "vulnerability_class": [
      "Improper Authorization"
    ],
    "source": "https://semgrep.dev/r/python.lang.security.audit.dynamic-urllib-use-detected.dynamic-u",
    "shortlink": "https://sg.run/dKZZ"
  },
  "severity": "WARNING",
  "fingerprint": "requires login",
  "lines": "requires login",
  "validation_state": "NO_VALIDATOR",
  "engine_kind": "OSS"
}
}
}
```